

Práctica 2: Gestión *Excepciones* en Java

Ejercicio 1: Captura de múltiples excepciones no verificadas (*Unchecked*)

- **Objetivo:** Manejar diferentes tipos de excepciones numéricas y de entrada de datos mediante bloques catch múltiples.
- **Enunciado:** Pide dos números enteros por consola al usuario y realiza su división. Captura explícitamente InputMismatchException (si el usuario introduce texto en lugar de un número) y ArithmeticException (si el usuario intenta dividir entre cero), mostrando un mensaje de error personalizado para cada caso.

Ejercicio 2: Conversión de datos y captura de NumberFormatException (*Unchecked*)

- **Objetivo:** Procesar arrays de texto convirtiéndolos a números sin interrumpir la ejecución ante datos corruptos.
- **Enunciado:** Dado un array de cadenas de texto con valores numéricos y algunos datos erróneos (por ejemplo, {"100", "250", "30A", "400", "XYZ"}), recorre el array convirtiendo cada elemento a entero (Integer.parseInt). Muestra cada número convertido y, si se detecta un valor no válido, captura la excepción NumberFormatException, muestra un aviso con el índice del elemento corrupto y continúa procesando el resto del array.

Ejercicio 3: Creación de ficheros y captura de IOException (*Checked*)

- **Objetivo:** Manejar la excepción verificada (IOException) que obliga a capturar el método createNewFile() de la clase File.
- **Enunciado:** Crea un programa que intente crear un fichero llamado documento.txt dentro de una subcarpeta llamada carpeta_inexistente. Al no existir previamente dicha carpeta contenedora, el método createNewFile() fallará y lanzará una excepción. Captura la excepción mediante un bloque try-catch para mostrar un mensaje de error explicativo sin que la aplicación falle.

Ejercicio 3: Resolución de rutas con getCanonicalPath() y bloque finally (*Checked*)

- **Objetivo:** Capturar la excepción verificada (IOException) al consultar rutas canónicas en el sistema de archivos utilizando la clase File junto con el bloque finally.
- **Enunciado:** Crea una instancia de la clase File pasándole una ruta relativa con redundancias (por ejemplo, ./subcarpeta/./fichero.txt). Utiliza el método getCanonicalPath() para obtener la ruta limpia y resuelta del archivo. Captura la posible IOException obligatoria y añade un bloque finally que confirme por consola el término de la inspección.